RABBITMQ / .NET / IDEMPOTENCY

RabbitMQ redelivery in .NET
Dedupe before you process.

A consumer that rejects a redelivered message instead of reprocessing it.

OrderConsumer.cs / dedupe guard — CSHARP

```csharp
public async Task HandleAsync(BasicDeliverEventArgs ea, CancellationToken ct)
{
    var messageId = ea.BasicProperties.MessageId;
    var key = $"processed:{messageId}";

    var isNew = await cache.StringSetAsync(
        key, "1", TimeSpan.FromHours(24), When.NotExists);

    if (!isNew)
    {
        channel.BasicAck(ea.DeliveryTag, false);
        return;
    }

    await ProcessOrderAsync(ea.Body, ct);
    channel.BasicAck(ea.DeliveryTag, false);
}
```

NOTE

Brokers guarantee at-least-once delivery, not exactly-once. Track processed message IDs with a short TTL so a redelivery is a no-op, not a duplicate order.

Manjunath Krishnasa

Architecture Notes
manjunathhk.in /manjunathhk

EF CORE / POSTGRESQL / N+1

N+1 hides until traffic finds it
Query once, not per row

Loading an order list should not fire one query per order for its line items.

BEFORE / lazy navigation per order — CSHARP

```csharp
var orders = await db.Orders
    .Where(o => o.CustomerId == id)
    .ToListAsync(ct);

foreach (var o in orders)
{
    // fires one query per order
    var total = o.Items.Sum(i => i.Price);
}
```

AFTER / project what you need — CSHARP

```csharp
var orders = await db.Orders
    .Where(o => o.CustomerId == id)
    .Select(o => new OrderSummary(
        o.Id, o.Items.Sum(i => i.Price)))
    .ToListAsync(ct);
```

NOTE

Lazy navigation looks correct in a unit test with three rows and falls over at production scale. Project or Include so EF Core issues one query, not N+1.

Manjunath Krishnasa

Architecture Notes
manjunathhk.in /manjunathhk

NESTJS / .NET / VALIDATION

Same contract, two runtimes. Validate at the edge.

A CreateOrder DTO rejected before it reaches the domain layer, in Node and in .NET.

NestJS — TYPESCRIPT

```typescript
class CreateOrderDto {
  @IsUUID()
  customerId: string;

  @IsInt()
  @Min(1)
  quantity: number;
}
```

.NET minimal API — CSHARP

```csharp
public record CreateOrderDto(
    [property: Required] Guid CustomerId,
    [property: Range(1, int.MaxValue)] int Quantity
);
```

Manjunath Krishnasa

Architecture Notes
manjunathhk.in /manjunathhk

NGINX / RATE LIMITING

A limit that trips itself. limit_req needs burst.

Without burst and nodelay, a page load with six assets rate-limits itself.

No burst — NGINX ✕

```
limit_req_zone $binary_remote_addr
    zone=api:10m rate=5r/s;

location /api/ {
    limit_req zone=api;
}
```

- ✕ Six parallel requests trip the limit for one user
- ✕ 503s show up in real traffic, not just load tests

Zone + burst — NGINX ✓

```
limit_req_zone $binary_remote_addr
    zone=api:10m rate=5r/s;

location /api/ {
    limit_req zone=api burst=12 nodelay;
}
```

- ✓ Absorbs a page's worth of concurrent calls
- ✓ Still caps sustained abuse at 5r/s

NOTE

A tight rate without burst rejects the same client's own concurrent asset requests, not just abusive traffic. Zone size and burst are what make the limit usable.

Manjunath Krishnasa

Architecture Notes
manjunathhk.in /manjunathhk

DOCKER / COMPOSE / HEALTHCHECKS

Healthy doesn't mean started depends_on: service_healthy

Four services, four healthchecks — the API waits for real readiness, not just a running container.

postgres YAML

```yaml
postgres:
  image: postgres:16
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U app"]
    interval: 5s
    retries: 5
```

redis YAML

```yaml
redis:
  image: redis:7
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 5s
    retries: 5
```

rabbitmq YAML

```yaml
rabbitmq:
  image: rabbitmq:3-management
  healthcheck:
    test: ["CMD", "rabbitmq-diagnostics", "ping"]
    interval: 10s
    retries: 5
```

api YAML

```yaml
api:
  build: .
  depends_on:
    postgres:
      condition: service_healthy
    redis:
      condition: service_healthy
```

NOTE

condition: service_started only waits for the container to launch. service_healthy waits for the healthcheck to pass, which is what dependent services actually need.

Manjunath Krishnasa

Architecture Notes
manjunathhk.in /manjunathhk

ANGULAR / DI / SIGNALS

Same DI, less ceremony. inject() in Angular

Constructor injection still works; inject() drops the boilerplate.

Constructor injection — TYPESCRIPT

```ts
export class OrderListComponent {
  constructor(
    private orders: OrderService,
    private auth: AuthService,
  ) {}
}
```

- Verbose for components with several dependencies
- Parameter order matters for readability

inject() — TYPESCRIPT

```ts
export class OrderListComponent {
  private orders = inject(OrderService);
  private auth = inject(AuthService);
}
```

- Reads top to bottom as plain field assignment
- Works in functional guards and resolvers too

NOTE

inject() only works in an injection context — a constructor, field initializer, or factory — not a later callback.

Manjunath Krishnasa

Architecture Notes
manjunathhk.in /manjunathhk

CSS / DESIGN TOKENS

Name the role, not the colour. Semantic design tokens

A stylesheet that says "accent" picks up the next palette release; one that says #2a5aa8 never will.

Raw values — CSS

```css
.button {
  background: #2a5aa8;
  color: #ffffff;
  border: 1px solid #87816f;
}
```

Semantic tokens — CSS

```css
.button {
  background: var(--mk-color-accent);
  color: var(--mk-color-on-accent);
  border: 1px solid var(--mk-color-border-control);
}
```

NOTE

Primitives hold raw values and never reach CSS; semantic tokens hold meaning and are all a site sees. Renaming one breaks consumers, changing its value does not.

Manjunath Krishnasa

Architecture Notes
manjunathhk.in /manjunathhk

CSS / DARK MODE

Follow the OS, allow an override
Light and dark from one file

prefers-color-scheme picks the mode by default; a data-theme attribute on <html> forces one.

tokens.css CSS

```css
:root {
  --mk-color-bg: #F2F0EB;
  color-scheme: light;
}

@media (prefers-color-scheme: dark) {
  :root:not([data-theme="light"]) {
    --mk-color-bg: #0F141B;
    color-scheme: dark;
  }
}

:root[data-theme="dark"] {
  --mk-color-bg: #0F141B;
  color-scheme: dark;
}
```

- No attribute: the OS preference decides
- color-scheme switches native controls too

index.html HTML

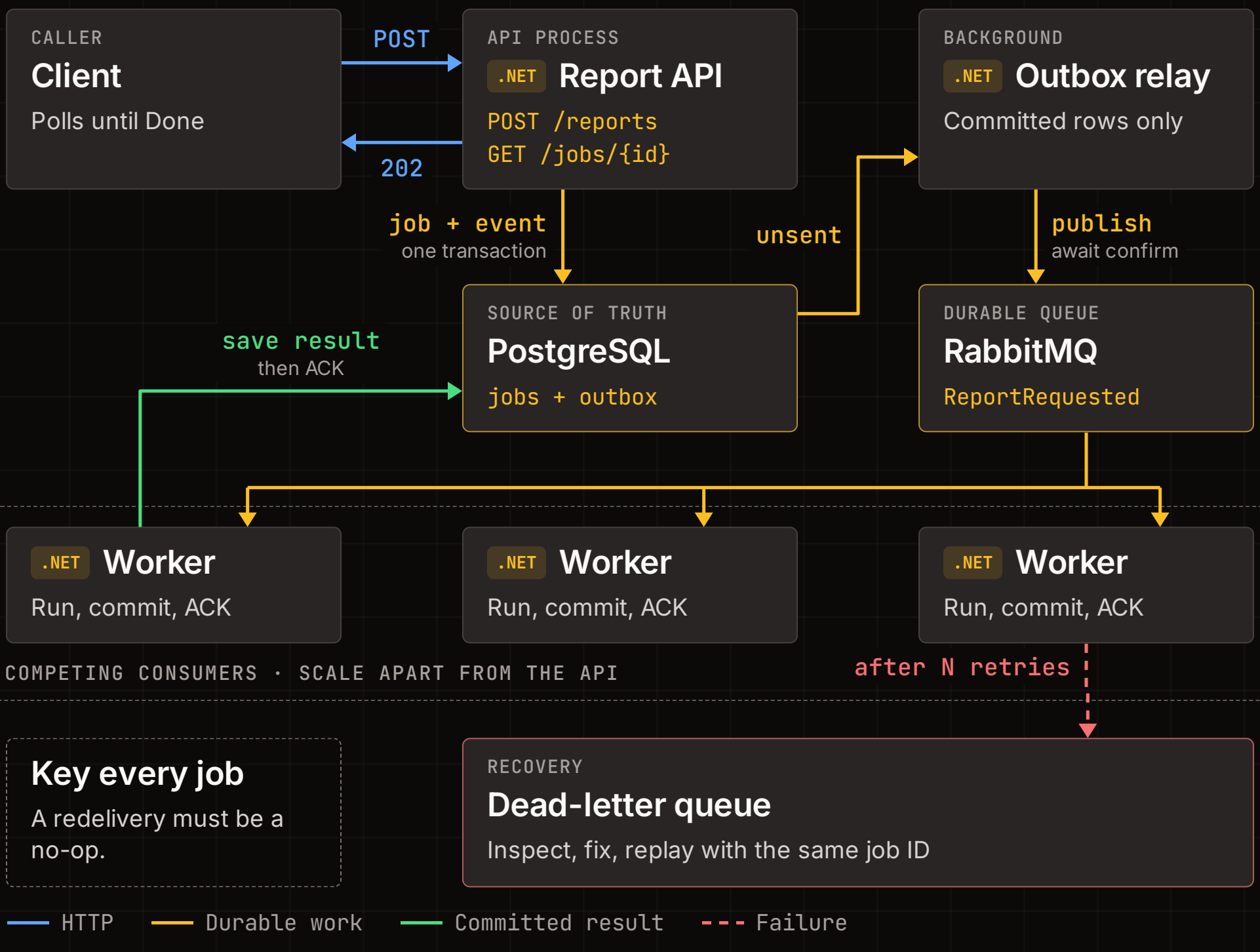
```html
<html data-theme="dark">
  <head>
    <link
      rel="stylesheet"
      href="/v1/index.css"
    />
  </head>
  <body>
    …
  </body>
</html>
```

- data-theme="light" or "dark" forces a mode
- Remove it to follow the OS again

.NET / RABBITMQ / BACKGROUND WORK

Scale the workers, keep the API.

Return 202 with a job ID. Let separate workers do the slow part.



NOTE

Watch queue age, not CPU. A growing backlog means you need more workers, not more copies of the API.

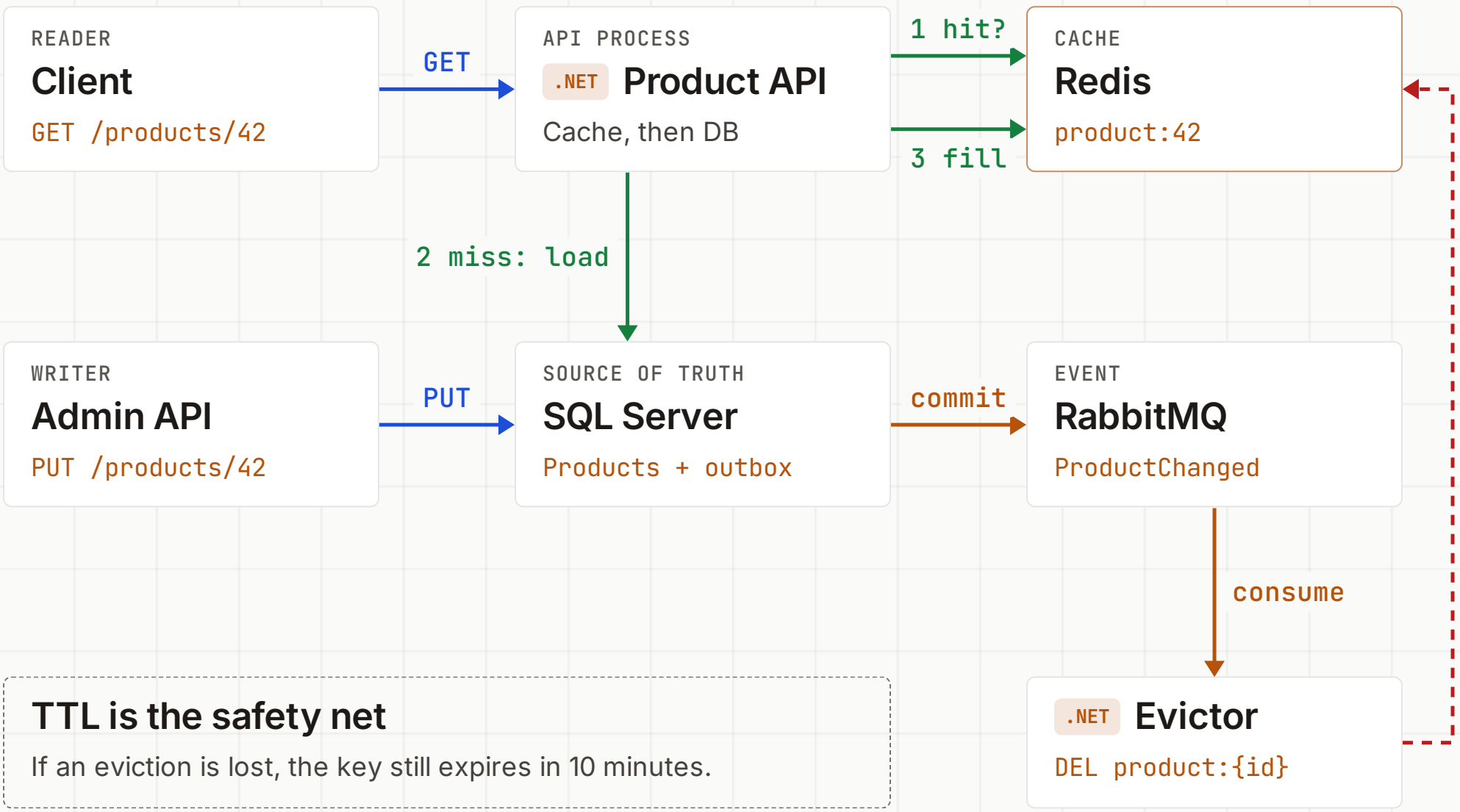
Manjunath Krishnasa

Architecture Notes
manjunathhk.in /manjunathhk

REDIS / SQL SERVER / CACHING

Cache-aside, evict on commit.

Reads fill Redis on a miss. Writes evict through an event, so the cache never outlives its row.



NOTE

Evict, never update the cached value. The next read refills from the source of truth, so two racing writers cannot leave a stale copy behind.

Manjunath Krishnasa

Architecture Notes
manjunathhk.in /manjunathhk